System Architecture & Technical Specification: Enochian Phonetic TTS & Closed-Vocabulary ASR Engine

1. Project Overview & Objective

An avant-garde audio pipeline designed to synthesize, process, and transcribe Enochian ceremonial and linguistic constructs through deterministic phonetic modeling.
The system operates across two primary phases:

- **Phase 1 (Synthesis):** Generate high-fidelity Text-to-Speech (TTS) for authentic historical Enochian vocabulary, as well as arbitrary English words transliterated into Enochian phonetic space via Hebrew Cabbalistic intermediaries.
- **Phase 2 (Recognition):** Capture Enochian audio and perform Speech-to-Text (STT) decoding back into original Enochian lexical roots and their English semantic translations.

2. Theoretical Validation & Architectural Constraints

Core Finding: Off-the-Shelf STT Failure Modes

Standard commercial ASR systems (OpenAI Whisper, Google Cloud Speech, Deepgram) couple an acoustic encoder with an autoregressive English Language Model (LM).

- **The Failure:** When presented with out-of-vocabulary Enochian phonemes, the LM forces predictions toward high-probability English tokens (e.g., *Iad Balt* → *"eyed belt"* or *"odd bolt"*).
- **The Solution:** Decouple acoustic frame decoding from semantic interpretation. Use an acoustic-only model (CTC-based) to output raw International Phonetic Alphabet (IPA) tokens, followed by a weighted lexical alignment layer against the finite Enochian corpus.

3. System Architecture





4. Linguistic & Phonetic Mapping Rules

Phonetic Traditions (G2P Engine)

The system must support two togglable pronunciation profiles:

1. **Historical Dee / Early Modern English (EME):**
   - Based on John Dee's marginal notes (Sloane MS 3188/3191) as cataloged by Aaron Leitch (*The Angelical Language*).
   - Naturalistic vowel shifts: *I/Y* as /j/ or /iː/; *CH* as velar fricative /x/ or plosive /k/; *C* hard before back vowels, soft before front vowels.
   - Example: *Iad Balt* → /jɑːd bɑːlt/.
2. **Hermetic Order of the Golden Dawn / Thelemic (*Liber Chanokh*):**
   - Staccato, syllable-expanding ritual vocalization.
   - Consonant isolation via epenthetic vowel insertion: clusters are forced open using short

/ɛ/ or /ə/.
    - Consonant transformations: *Z* vocalized as *Zod* (/zɒd/); terminal releases aspirated.
    - Example: *Iad Balt* → /iː.ɑːd bɑːl.tə/.

Intermediate Hebrew Consonant Bridge

For Milestone 2 (synthesizing arbitrary English concepts into Enochian phonetic space):

- **Consonant Mapping:** Transliterate the target word into standard Hebrew abjad representation (e.g., *Choronzon* → ן-ו-ז-נ-ו-ר-ו-ם).
- **Occult Attribution Matrix:** Map the 21 historical Enochian characters to the 22 Hebrew paths via the Golden Dawn elemental/astrological schema (e.g., Cheth → *Graph*, Resh → *Don*, Nun → *Drux*, Zayin → *Ceph*).
- **Vowel Construction:** Treat *Vav* and *Yod* as *matres lectionis* (/oʊ/, /uː/, /iː/), with automated schwa (/ə/) insertion between unvocalized consonant clusters.

5. Implementation Roadmap & Technical Deliverables

Phase 1: Phonetic TTS Engine

1. **Lexicon Database (lexicon.json):**
    - Build a structured dictionary containing Dee's historical corpus (~1,000–1,200 unique roots from the 48 Keys, 30 Aethyrs, and Great Table).
    - Fields: root, dee_ipa, gd_ipa, hebrew_skeleton, english_meaning.
2. **Algorithmic G2P Engine (enochian_g2p.py):**
    - Fallback rule-engine for novel/transliterated words.
    - Converts raw Enochian or transliterated Hebrew letter chains into valid IPA/X-SAMPA strings.
3. **TTS Output Pipeline:**
    - Support SSML output generation (<phoneme alphabet="ipa" ph="...">) for high-end neural cloud TTS.
    - Provide offline synthesis hooks via espeak-ng or local neural vocoders (e.g., Piper, Coqui TTS).

Phase 2: Acoustic Extraction & Lexical Alignment

1. **Phoneme Extractor (audio_to_ipa.py):**
    - Integrate Hugging Face transformers using an acoustic-only CTC model (e.g., facebook/wav2vec2-lv-60-espeak-cv-ft).
    - Takes 16 kHz mono WAV input; produces a pure frame-level IPA symbol stream without applying an English vocabulary bias.
2. **Abjad Normalizer & Weighted Matcher (aligner.py):**
    - Strips transient acoustic noise and minor vowel variations.
    - Uses weighted Levenshtein distance against the lexicon.json phonetic index.
    - Consonant-heavy scoring: penalizes consonant mismatches heavily while tolerating dialectal vowel drift (e.g., /ɑː/ vs. /æ/).
3. **Decoder & Translator (enochian_decoder.py):**
    - Resolves the matched token to its canonical Enochian root.
    - Emits the corresponding English translation (or reverses the Hebrew-English

transliteration for modern phrases).